

Optimisation de la réduction d'expressions du lambda calcul

Hurot Eliott

2025-2026

Définitions 2

 Syntaxe & Sémantique 3

 Réductions 4

α -réduction 4

β -réduction 4

 Expressivité et sucre 5

 Récursivité 6

Terminaison et confluence 7

 Terminaison 8

 Confluence 9

Stratégies de réductions 10

 Stratégies internes 11

 Stratégies externes 12

 LE Théorème 13

Programme et Optimisation 14

 Indice de De Bruijn 15

 Algorithme de recherche de plus cours chemin 16

 File de priorité 17

 Heuristique 18

Définitions

$\Lambda := x$ Variable
| MN Application
| $\lambda x.M$ Fonction

$M, N \in \Lambda$ et $x \in V$

Rédex : $(\lambda x.M)N$

α -réduction

Renommage

$$\lambda x.x + 1 \stackrel{?}{=} \lambda y.y + 1$$

$$\lambda x.u \quad \alpha \quad \lambda y.(u[x := y]) \quad \Rightarrow \quad =_{\alpha} \quad \Rightarrow \quad \Lambda / =_{\alpha}$$

β -réduction

Exécution

$$(\lambda x.u)v \quad \beta \quad u[x := v] \quad \Rightarrow \quad \rightarrow$$

Passe au contexte :

$$\left| \begin{array}{l} u_1 \stackrel{=}{\alpha} u_2 \\ v_1 \stackrel{=}{\alpha} v_2 \end{array} \right. \Rightarrow u_1 v_1 \stackrel{=}{\alpha} u_2 v_2$$

$$u \stackrel{=}{\alpha} v \Rightarrow \lambda x.u \stackrel{=}{\alpha} \lambda x.v$$

Theorem 1 (Equivalence fonction récursive / lambda terme)

Pour toute fonction récursive f , il existe un λ -terme $\lceil f \rceil$ qui code correctement f
 $\forall m = (m_1, \dots, m_k) \in \mathbb{N}^k / f(m)$ est défini,

$$\lceil f \rceil(\lceil m_1 \rceil, \dots, \lceil m_k \rceil) \equiv_{\beta} \lceil f(m) \rceil$$

Exemples :

$$\forall n \in \mathbb{N}, \lceil n \rceil := \lambda f, x. f^n x \quad , \quad f^0 t := t \quad f^{n+1} t := f(f^n t)$$

$$\lceil S \rceil := \lambda n, f, x. f(n f x)$$

$$\lceil + \rceil := \lambda m, n, f, x. m f(n f x)$$

$$\lceil V \rceil := \lambda x, y. x$$

$$\lceil F \rceil := \lambda x, y. y$$

$$\lceil < u, v > \rceil := \lambda z. z u v$$

$$\llbracket \text{fact} \rrbracket \stackrel{\beta}{=} \lambda n. \llbracket \text{if} \rrbracket (\llbracket \text{zero ?} \rrbracket n) \llbracket 1 \rrbracket ((\llbracket \times \rrbracket n (\llbracket \text{fact} \rrbracket (\llbracket P \rrbracket n)))$$

$$\llbracket \text{fact} \rrbracket = F \llbracket \text{fact} \rrbracket$$

Theorem 1 (Existence de point fixe)

Tout λ -terme F a un point fixe x , i.e. $x \stackrel{\beta}{=} Fx$

Theorem 2 (Combinateur de point fixe)

Il existe un λ -terme Y sans variable libre tel que pour tout F , YF soit un point fixe de F .

Proof. Prendre $Y := \lambda f. (\lambda x. f(xx)) (\lambda x. f(xx))$

□

Terminaison et confluence

Forme normale : on ne peut plus simplifier $\Rightarrow$ Unicité / Existence ?

La β réduction ne termine pas toujours : $\Omega := (\lambda x.xx)(\lambda x.xx)$

$$\Omega \rightarrow (xx)[x := (\lambda x.xx)] \rightarrow (\lambda x.xx)(\lambda x.xx) = \Omega$$

Fortement normalisant : LES réductions se terminent

Faiblement normalisant : UNE Réduction se termine

$$(\lambda x.y)\Omega$$

Theorem 1 (Confluence du lambda calcul)

Si $u \xrightarrow{*} v_1$ et $u \xrightarrow{*} v_2$, alors il existe w tel que $v_1 \xrightarrow{*} w$ et $v_2 \xrightarrow{*} w$

Corollary 1.1 (Unicité des formes normales)

Si u_1 et u_2 sont deux formes normales de u , alors $u_1 = u_2$

Stratégies de réductions

```
int incr(int x){  
    return x + 1;  
}  
int main(){  
    int x = 4;  
    int y = incr(x);  
}
```

```
let a = ref 0  
let f x y = ()  
let () = f (a:=1) (a:=2);  
print_int !a
```

$$\begin{aligned} uv &\rightarrow u'v \rightarrow u'v' \\ &\rightarrow (\lambda x.t)v \rightarrow (\lambda x.t)v' \rightarrow t[x := v'] \end{aligned}$$

$$(\lambda x.u)v \rightarrow u[x := v]$$

$$[\text{fact}]([+][1][2])$$

$$\xrightarrow{*} (\lambda n. [\text{if}]([\text{zero ?}][n])[1]([\times][n]([\text{fact}]([P][n]))))([+][1][2])$$

$$\rightarrow [\text{if}]([\text{zero ?}]([+][1][2]))[1]([\times]([+][1][2])([\text{fact}]([P]([+][1][2]))))$$

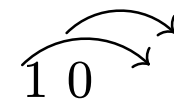
$$[\text{fact}]([+][1][2])$$

$$\xrightarrow{*} [\text{if}]([\text{zero ?}][3])[1]([\times][3]([\text{fact}]([P][3])))$$

Theorem 1 (Standardisation)

Si u est faiblement normalisant, alors la stratégie externe gauche calcule la forme normale de u par une réduction finie.

Programme et Optimisation

$$\lambda x. \lambda y. xy = \lambda. \lambda. 1 \ 0$$


$[P][10]$: Ordre : 50551 | Taille : 310915

